

[Repository](#)

Background context

BufANN is organized around a graph-based dynamic ANN index plus a lightweight metadata and workload pipeline. We will implement IVF-PQ as a separate index type with its own on-disk page format, built and deployed independently of the graph-based index. The choice between BufANN graph index or an IVF-PQ index is made at startup.

How IVF-PQ fits with Buffer Pool Management

- Which index type to build is chosen at build/startup time via a config option (e.g. `index_type=ivf_pq` vs. `index_type=graph`), with IVF-specific parameters (`ivf_nprobe`, `ivf_nlist`, `pq_chunks`) for when `ivf_pq` is chosen.
- IVF-PQ carries no graph adjacency and modifies BufANN's `PackedSlotHeader` and page layout. Raw vectors are stored in a dense, fixed-position array.
- generate centroids via k-means, assign vectors to partitions, train PQ pivots, encode all vectors with PQ, build posting lists, and persist all of this.

Implementation Details

Data Structures

```
struct IVFMetadata {
    uint32_t nlist;
    uint32_t dim;
    uint32_t aligned_dim;
    std::vector<float> centroids; // shape: [nlist, aligned_dim]
};
```

```
struct ClusterAssignments {
    std::vector<uint32_t> cluster_id;
    // shape: [N], cluster_id[i] = centroid index for vector i
};
```

```
struct PostingLists {
    std::vector<uint32_t> offsets;
    std::vector<uint32_t> ids;
};
```

offsets[i] = start of partition i in ids array

ids is an array of all point IDs such that partition i = ids[offsets[i]:offsets[i+1]]

```
struct PQMetadata {
    uint32_t chunks;
    uint32_t chunk_dim;
    uint32_t k; // number of PQ centers per chunk (256 for uint8)
    std::vector<float> pivots; // shape: [chunks, k, chunk_dim]
    std::vector<uint8_t> codes; // shape: [N, chunks]
};
```

```
struct IVFPQSearchConfig {
    bool is_ivf_pq_index;
    uint32_t nprobe;
    std::atomic<const IVFMetadata*> ivf_meta;
    std::atomic<const ClusterAssignments*> assignments;
    std::atomic<const PostingLists*> posting_lists;
    std::atomic<const PQMetadata*> pq_meta;
};
```

We use atomics to allow us to atomically swap these pointers after background threads finish building rebuilds for PostingLists (or ClusterAssignments). We don't want rebuilds to hurt P99(9) latency, so searches must be able to run at the same time as rebuilds.

IVF-PQ Memory Footprint and Sidecar Persistence Layout

Assuming 1e9 vectors and 32 PQ chunks (tunable down if memory becomes tight), IVF-PQ's full metadata (centroids, cluster assignments, posting lists, PQ pivots, and PQ codes) is small enough to load entirely into memory.

PQ codes already exist as an in-memory array in BufANN today (~32 GB at this scale). IVF-PQ adds 5 more structures on top: centroids (~0.5 GB), PQ pivots (negligible), posting-list offsets/IDs (~4 GB), RID table (~4 GB), and cluster assignments (~4 GB), totaling an additional ~12.5 GB.

```
struct RawVectorRID {
    uint32_t packed;
};
```

- `uint32_t flat_slot = packed & 0x7FFFFFFFu;`
 `bool active = (packed >> 31) != 0;`
 `uint32_t page_id = flat_slot / 7;`

```
uint32_t slot_idx = flat_slot % 7;
```

- We pack this information into 32 bits as the active boolean is 1 bit, and the slot id is 3 bits.

```
struct RawVectorRIDTable {  
    std::vector<RawVectorRID> rid;  
};
```

We need to store the locations of raw vector data on disk, as inserts are not necessarily ordered (we can't determine location with only ID).

```
struct IVFPQIndexFileHeader {  
    uint32_t magic;  
    uint32_t version;  
    uint32_t nlist;  
    uint32_t dim;  
    uint32_t aligned_dim;  
    uint32_t pq_chunks;  
    uint32_t pq_chunk_dim;  
    uint32_t pq_k;  
    uint64_t num_vectors;  
    uint64_t centroids_offset;  
    uint64_t centroids_bytes;  
    uint64_t cluster_assignments_offset;  
    uint64_t cluster_assignments_bytes;  
    uint64_t posting_offsets_offset;  
    uint64_t posting_offsets_bytes;  
    uint64_t posting_ids_offset;  
    uint64_t posting_ids_bytes;  
    uint64_t pq_pivots_offset;  
    uint64_t pq_pivots_bytes;  
    uint64_t pq_codes_offset;  
    uint64_t pq_codes_bytes;  
    uint64_t rid_table_offset;  
    uint64_t rid_table_bytes;  
    uint32_t raw_vector_elem_size;  
    uint32_t raw_vector_page_size;  
    uint64_t raw_vectors_bytes;  
};
```

Raw-vector page layout (per page, 4096 B, dim=128/fp32):

Field	Size	Offset
Page header (version/reserved)	8 B	0
Occupancy bitmap (7 slots)	1 B	8
Vector slot 0	512 B	9
Vector slot 1	512 B	521
Vector slot 2	512 B	1033
Vector slot 3	512 B	1545
Vector slot 4	512 B	2057
Vector slot 5	512 B	2569
Vector slot 6	512 B	3081
(unused)	503 B	3593

```
struct RawVectorFreeList {
    std::mutex mtx;
    std::vector<uint32_t> free_slots;
};
```

We maintain a free list similar to the graph search index.

```
struct PostingListDelta {
    std::unordered_map<uint32_t, std::vector<uint32_t>> pending_inserts;
    tsl::robin_set<uint32_t> tombstones;
};
```

We also maintain a delta buffer.

Building IVF-PQ

IVF-PQ's own buffer pool does not manage anything until we are done building IVF-PQ, as the raw-vector heap is written directly with bulk sequential writes.

1. Train centroids:
 - sample or stream k-means on base vectors
 - persist centroids
2. Assign vectors to clusters:
 - for each vector: $\text{cluster_id}[i] = \underset{j}{\operatorname{argmin}} ||\text{vector}[i] - \text{centroid}[j]||^2$
 - allocate a page/slot in the raw-vector heap, write the raw vector bytes, record RawVectorRID
 - persist cluster assignments and the RID table
3. Build posting lists:
 - group vector IDs by cluster_id

- create offsets and flat IDs array
- persist
- 4. Train PQ pivots:
 - for each chunk c:
 - extract all values from dimension range $[c \cdot \text{chunk_dim}, (c+1) \cdot \text{chunk_dim})$
 - run k-means with $k=256$ centers
 - persist pivots
- 5. Encode vectors with PQ:
 - for each vector:
 - for each chunk c:
 - $\text{code}[i, c] = \underset{j}{\text{argmin}} \ ||\text{vector}[i, \text{chunk_range}] - \text{pivot}[c, j]||^2$
 - persist codes
- 6. Create IVF-PQ index metadata file:
 - nlist, dim, aligned_dim, chunks, chunk_dim, k
 - offsets to sidecar: centroids, cluster assignments, posting lists, pivots, PQ codes, RID table
 - raw-vector heap descriptor: element size, page size, total byte count

Querying IVF-PQ

```
float distance_table[chunks][256];
for (uint32_t c = 0; c < chunks; c++) {
    for (uint32_t p = 0; p < 256; p++) {
        distance_table[c][p] = ||query_chunk[c] - pivot[c, p]||^2;
    }
}
float pq_distance(const uint8_t* vector_pq, const float** distance_table,
uint32_t chunks) {
    float dist = 0.0;
    for (uint32_t c = 0; c < chunks; c++) {
        dist += distance_table[c][vector_pq[c]];
    }
    return dist;
}
```

(purple = disk read/write)

1. Build the per-chunk PQ distance table:
 - for each chunk c, each pivot p: $\text{distance_table}[c][p] = ||\text{query_chunk}[c] - \text{pivot}[c, p]||^2$
2. Compute distances to all centroids:
 - $\text{distances} = ||\text{query} - \text{centroid}||^2$ for all nlist centroids
3. Select top nprobe centroids
4. Gather candidates:
 - for each selected partition p: $\text{candidates.extend}(\text{posting_ids}[\text{offsets}[p] : \text{offsets}[p+1]])$

- merge in pending inserts from the delta buffer and drop any pending-deleted IDs
5. Rank candidates using PQ:
- sort by $pq_dist = pq_distance(candidate_pq_code, distance_table, chunks)$
 - keep top M ($M > K$)
6. Re-rank the top-M candidates on exact distance
- for each candidate: look up $rid_table[candidate_id] \rightarrow (page_id, slot_idx)$; **read the raw vector data**, compute exact distance to the query
 - sort by exact distance; keep top K

```
std::unordered_map<uint32_t, std::vector<uint8_t>> pq_codes_dynamic;
```

pq_codes_dynamic will cache recent inserts.

Insert

Input: (vector, vector_id)

```
cluster_id ← argmin_c ||vector - centroid[c]||² for all c in [0, nlist)
for each chunk c in [0, chunks):
    pq_code[c] ← argmin_p ||vector[c*chunk_dim:(c+1)*chunk_dim] - pivot[c, p]||²
rid ← allocate_slot()
write vector bytes to rid.page_id/slot_idx
rid_table[vector_id] ← rid (active = 1)
posting_lists_delta[cluster_id].append(vector_id)
cluster_assignments[vector_id] ← cluster_id
pq_codes_dynamic[vector_id] ← pq_code
```

Delete

```
Input: vector_id
posting_lists_tombstones.add(vector_id)
posting_lists_delta[cluster_assignments[vector_id]].remove(vector_id)
rid ← rid_table[vector_id]
clear occupancy bit at rid.page_id/slot_idx
rid_table[vector_id].active ← 0
free_list.push(rid.page_id, rid.slot_idx)
if vector_id in pq_codes_dynamic:
    pq_codes_dynamic.erase(vector_id)
```

Streaming Out-of-Core K-Means For IVF Centroids

- Process small, random batches of data sequentially instead of loading the entire dataset at once.
- Update cluster centers by using a weighted average of the old center and new point

Re-clustering, repartitioning, or periodic re-clustering scheduling

Insertions and deletions change the distribution of vectors across partitions. If a few centroids become overloaded, the search path can degrade quickly even if the coarse quantizer itself is still valid.

The simplest initial/baseline design is to do no rebalancing. Partition quality will drift over time if the workload is skewed or there is a high volume of insertions/deletions.

We can evaluate IVF-PQ benchmarks without rebalancing, and we may consider rebalancing if performance degrades over time. If it's needed, it uses the same build-off-to-the-side, atomic pointer swap, grace-period reclaim mechanism already designed for the routine posting-list rebuild.

If needed later, we could choose to add:

- partition imbalance tracking
- optional rebuild triggers based on workload skew
- periodic re-clustering or re-encoding

Other considerations

Durability and crash recovery

- Periodically write the six flat structures (centroids, cluster assignments, posting lists, pivots, PQ codes, RID table) to disk and maintain an append log.
- The log entry is durable only once the corresponding page happens.
- Recovery: load the last snapshot, then replay the append-log entries written after it to reconstruct `posting_lists_delta`, `pq_codes_dynamic`, and tombstones.

Garbage Collection

- `DeleteSet.h` manually does garbage collection but isn't wired into the graph search index right now. We could implement something similar for IVF-PQ on top of the Buffer Pool Manager.
- Using `StrongPtr` loses concurrency as an atomic reference counted pointer uses a Spin Lock rather than an atomic instruction

Tuning

- `nlist` and `nprobe`
- `pq_chunks` (accuracy vs. memory/compute)
- `M` (re-rank oversample factor), number of vectors passed into exact distance and top-K
- Rebuild delta-threshold (gather latency as delta size grows)